UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 07 • **Ejercicio 03** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 3: Empresa de Logística y Envíos (Jerarquía de Clases Abstractas)

Contexto

En este ejercicio profundizaremos en el uso de **Clases Abstractas** (`abstract class`) como raíz de una jerarquía conceptual ("es-Un") que posee estado protegido compartido, constructores con validación de invariantes de negocio y métodos concretos que delegan en métodos abstractos.

A diferencia del Ejercicio 2 (donde los medios de pago eran contratos de comportamiento puro sin atributos ni invariantes compartidas), todos los envíos gestionados por una empresa de distribución son inequívocamente paquetes con identidad, peso y precio base comunes, exigiendo una clase abstracta que evite la duplicación de código y centralice la consistencia del dominio.

Enunciado

Una empresa nacional de paquetería y logística requiere desarrollar el módulo central de tarifación para su flota de distribución de encomiendas.

Todos los envíos gestionados por la empresa comparten atributos esenciales y se modelan a partir de la clase abstracta `Envio` :

- **Atributos protegidos:** `codigoRastreo: String` , `pesoKg: double` y `precioBase: double` .

- **Constructor protegido:** Debe inicializar los atributos y garantizar las invariantes de negocio: el `pesoKg` debe ser estrictamente mayor a 0 y el `precioBase` no puede ser negativo. Ante valores inválidos, debe arrojar una excepción `IllegalArgumentException`.
- **Método concreto:** `public double calcularCostoTotal()`, el cual retorna la suma del `precioBase` más el recargo adicional obtenido mediante el método abstracto: `public abstract double calcularRecargoAdicional();`
- **Métodos accesores públicos:** `getCodigoRastreo()`, `getPesoKg()` y `getPrecioBase()`.

Las modalidades de entrega especializan el cálculo del recargo adicional según sus propias variables operativas:

1. **Envío Estándar (`EnvioEstandar`):** Modalidad terrestre económica. Posee un atributo propio `costoPorKg: double`. Su recargo adicional se calcula multiplicando el peso del paquete por dicho costo (`pesoKg * costoPorKg`).
2. **Envío Express (`EnvioExpress`):** Modalidad prioritaria con entrega en 24 horas. Posee `costoPorKg: double` y una tasa fija por urgencia `tasaPrioritaria: double`. Su recargo adicional es: `(pesoKg * costoPorKg) + tasaPrioritaria`.
3. **Envío Internacional (`EnvioInternacional`):** Modalidad aérea que cruza fronteras aduaneras. Posee `costoPorKg: double` y una `alicuotaAduanera: double` (porcentaje tributario sobre el precio base, ej. 0.20 para un 20%). Su recargo adicional es: `(pesoKg * costoPorKg) + (precioBase * alicuotaAduanera)`.

Por último, se debe implementar la clase `EmpresaLogistica`, la cual gestiona una colección `List<Envio> enviosRegistrados` y provee los siguientes servicios:

- `agregarEnvio(Envio e)`: registra un nuevo envío en el sistema (validar que no sea nulo).
- `calcularFacturacionTotal(): double`: itera polimórficamente sobre todos los envíos registrados y retorna la sumatoria de sus costos totales (`calcularCostoTotal()`).
- `obtenerEnviosPesados(double umbralPeso): List<Envio>`: retorna una lista filtrada con aquellos envíos cuyo peso supere el umbral indicado.

Tareas:

1. Diseñe la clase abstracta `Envio` con sus atributos protegidos, constructor con validación de invariantes, accesores, método abstracto `calcularRecargoAdicional()` y método concreto `calcularCostoTotal()`.
2. Implemente las subclases `EnvioEstandar`, `EnvioExpress` y `EnvioInternacional`, invocando a `super(...)` en sus constructores y sobreescribiendo adecuadamente el método abstracto.
3. Diseñe la clase `EmpresaLogistica` conteniendo la colección de envíos y sus operaciones polimórficas de facturación y filtrado.

Restricciones y Tips

- **Prohibición de Instanciación:** Compruebe que la clase `Envio` no pueda instanciarse directamente (`new Envio(...)`).
- **Constructor en Clase Abstracta:** Recuerde que, aunque no se pueda instanciar directamente, la clase abstracta debe poseer un constructor para centralizar la asignación y validación de las invariantes compartidas mediante `super(...)`.
- **Reutilización y Delegación:** El método `calcularCostoTotal()` debe estar implementado una única vez en `Envio`, delegando el recargo específico en `calcularRecargoAdicional()`. Ninguna subclase debe redefinir `calcularCostoTotal()`.
- **Programación hacia Abstracciones (OCP):** La clase `EmpresaLogistica` debe operar exclusivamente contra el tipo aparente `Envio`. Si la empresa incorpora nuevas modalidades (ej. `EnvioRefrigerado`), no debe ser necesario modificar su lógica de facturación o filtrado.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. Si una clase abstracta no se puede instanciar directamente con la palabra reservada `new`, ¿por qué resulta indispensable que posea un

constructor protegido?

2. ¿Por qué en este escenario de encomiendas es conceptualmente más adecuado utilizar una clase abstracta (`Envio`) en lugar de una interfaz pura como en el Ejercicio 2 (`MedioDePago`)?
3. ¿De qué manera la estructura del método `calcularCostoTotal()` en `Envio` promueve la reutilización de código y previene la duplicación de algoritmos en las subclases?